PRE-MIDTERM LECTURE

# Divide and Conquer

Merge Sort • Quick Sort • Maximum Subarray bridge

CORE IDEA

**split → solve**

multiple smaller

subproblems

**combine → answer**

# From Decrease and Conquer to Divide and Conquer

The key change is the number of smaller instances we must solve.

### DECREASE AND CONQUER

Solve ONE smaller instance.

Examples: insertion sort, binary search, Euclid.

The smaller solution is extended to solve the original problem.

### DIVIDE AND CONQUER

Split the original problem into TWO OR MORE smaller instances.

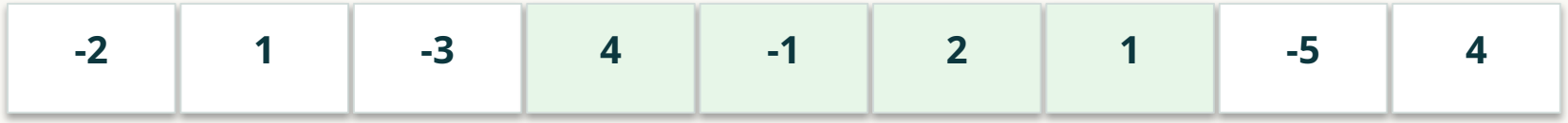
Solve the subproblems, usually recursively.

Combine the subproblem solutions into the final answer.

**ONE smaller instance → MULTIPLE smaller instances**

# A problem that exposes the difference

Find the contiguous subarray with the largest sum.




**Best subarray: [4, −1, 2, 1] → maximum sum = 6**

**THE QUESTION**

How should we design the algorithm—not merely how should we code it?

# Maximum Subarray by brute force

### GENERATE CANDIDATES

Choose every possible start index i and end index j.

### EVALUATE

Compute the sum of each candidate subarray.

### KEEP THE BEST

Remember the maximum sum seen so far.

**Number of candidate subarrays = $n(n+1)/2 = \Theta(n^2)$**

If every candidate sum is recomputed from scratch, the straightforward version can reach $\Theta(n^3)$.

# Why decrease-and-conquer is not the natural fit

### TRY REMOVING ONE ELEMENT

Which element should be removed?

The maximum subarray may lie entirely before it, after it, or include the region affected by the removal.

One smaller instance does not naturally preserve all possibilities.

### A BETTER STRUCTURAL QUESTION

What if we split the array around the middle?

Then every optimal subarray must be in one of three locations.

That structure gives us a natural recursive decomposition.

**The problem is telling us to solve more than one smaller instance.**

# Split the array in half




### LEFT HALF

Best subarray lies completely in the left half.

### RIGHT HALF

Best subarray lies completely in the right half.

### CROSSING

Best subarray crosses the midpoint.

**Answer = max(left best, right best, crossing best)**

# This becomes Divide and Conquer

$$T(n) = 2T(n/2) + \Theta(n)$$

**DIVIDE**

Split around the midpoint.

**CONQUER**

Recursively solve left and right halves.

**COMBINE**

Compute the best crossing subarray and take the maximum.

**Master Theorem → $\Theta(n \log n)$**

# Learning outcomes

## EXPLAIN

Divide and Conquer as a design strategy.

Trace Merge Sort and Quick Sort step by step.

Derive and interpret their recurrences.

## ANALYZE

Best, average and worst-case behavior.

Compare Merge Sort and Quick Sort.

Justify algorithm choice for a situation.

# Divide and Conquer — the core idea

### DIVIDE

Break the original problem into smaller subproblems.

### CONQUER

Solve the smaller subproblems, usually recursively.

### COMBINE

Join the subproblem solutions into the final answer.

**Base case: solve very small instances directly.**

# When does Divide and Conquer fit well?

### SPLIT

The problem can be divided into smaller instances of the same problem.

### INDEPENDENCE

Subproblems can be solved independently or nearly independently.

### COMBINE

Combining the answers is cheaper than solving the original problem directly.

# The three phases

### DIVIDE

Choose how to split the input.

### CONQUER

Recursively solve each smaller part.

### COMBINE

Construct the solution to the original problem.

**The quality of the split and the cost of combining largely determine performance.**

# From algorithm to recurrence

$$T(n) = aT(n/b) + f(n)$$

**A**

Number of recursive subproblems.

**B**

Shrink factor for each subproblem.

**F(N)**

Work done outside recursive calls.

The recurrence describes the algorithm; Master Theorem analyzes suitable recurrences.

# Bridge from brute force

### BRUTE-FORCE INVERSION COUNTING

Checks every pair → $\Theta(n^2)$.

Direct, systematic, but ignores structure.

### DIVIDE-AND-CONQUER THINKING

Sort each half.

Count inversions within each half.

Count cross-inversions while merging.

**Structure can reduce repeated work.**

# Merge Sort — what problem are we solving?




**Input:** an unsorted array of n values.

**Output:** the same values in nondecreasing order.

**Example:** [8,3,6,2,7,1,5,4] → [1,2,3,4,5,6,7,8]

# Merge Sort — algorithm

### BASE CASE

If the subarray has one element, it is already sorted.

### DIVIDE + CONQUER

Split into two halves and recursively sort both halves.

### COMBINE

Merge the two sorted halves.

**MERGESORT(A) → split → MERGESORT(left) → MERGESORT(right) → MERGE**

# Merge Sort — split all the way down

**[8, 3, 6, 2, 7, 1, 5, 4]**

↙️ ↘️

**[8, 3, 6, 2]** **[7, 1, 5, 4]**

↓ split until single-element arrays ↓

**[8] [3] [6] [2]** **[7] [1] [5] [4]**

**Single-element arrays are the base cases.**

# How MERGE combines two sorted lists

Merge [3, 8] and [2, 6].

Compare 3 and 2 → take 2.

Compare 3 and 6 → take 3.

Compare 8 and 6 → take 6.

Right list is exhausted → append 8.

Result: [2, 3, 6, 8]

**Each element is copied once during one merge → Θ(n) work at that level.**

# Merge Sort — combine back upward

[8] + [3] → [3,8]  [6] + [2] → [2,6]

[7] + [1] → [1,7]  [5] + [4] → [4,5]

[3,8] + [2,6] → [2,3,6,8]

[1,7] + [4,5] → [1,4,5,7]

Final merge → [1,2,3,4,5,6,7,8]

# Derive the Merge Sort recurrence

### DIVIDE

Midpoint calculation is Θ(1) in the simplified model.

### CONQUER

Two recursive calls, each on size n/2 → 2T(n/2).

### COMBINE

Merge two sorted halves → Θ(n).

$$T(n) = 2T(n/2) + \Theta(n)$$

**Therefore Merge Sort is Θ(n log n).**

# Merge Sort complexity — all cases

| Case | Time | Why |
|---|---|---|
| Best | Θ(n log n) | Still splits and merges every level |
| Average | Θ(n log n) | Same recursive structure |
| Worst | Θ(n log n) | Same recursive structure |

**Input order does not remove the merge work.**

# Merge Sort through Master Theorem

$$T(n)=2T(n/2)+\Theta(n)$$

**a =** 2

**b =** 2

**f(n) =** Θ(n)

**$\log_2$ a =** $\log_2 2 = 1$

**f(n) matches n^(log_b a) → equal-growth case → Θ(n log n)**

# Merge Sort practical properties

| Property | Merge Sort |
| --- | --- |
| Stable | Yes, when equal keys preserve relative order |
| Predictable | Θ(n log n) in best, average and worst cases |
| Extra memory | Typically Θ(n) for array-based merging |
| Useful when | Stable ordering or external/linked-list sorting matters |

# Quick Sort — the core idea

### CHOOSE PIVOT

Select one element as the pivot.

### PARTITION

Move smaller/equal values to one side and larger values to the other.

### RECURSE

Sort the left and right partitions.

**Unlike Merge Sort, the main work is in PARTITION rather than a final merge.**

# What does partitioning accomplish?

### BEFORE PARTITION

Values are mixed around the pivot.

The pivot does not yet have a guaranteed final position.

### AFTER PARTITION

Every value on one side satisfies the pivot rule.

The pivot is in its final sorted position.

The two sides do not need to be internally sorted yet.

**Partitioning only guarantees the pivot’s final position.**

# Lomuto PARTITION — simple version

Choose the last element as pivot.

i marks the end of the ≤ pivot region.

Scan j from low to high−1.

When A[j] ≤ pivot, advance i and swap A[i] with A[j].

Finally swap the pivot into position i+1.

Return the pivot index.

# QUICKSORT pseudocode

```
QUICKSORT(A, low, high)
   if low < high:
      p = PARTITION(A, low, high)
      QUICKSORT(A, low, p-1)
      QUICKSORT(A, p+1, high)
```

Base case: a subarray of size 0 or 1 is already sorted.

# Quick Sort trace — start




**Pivot:** 4

**Rule:** move values ≤ 4 to the left side and values > 4 to the right side.

**Goal:** pivot 4 reaches its final sorted position after partitioning.

# Partition around pivot 4 — first scan

Start: [8, 3, 6, 2, 7, 1, 5, 4]

8 > 4 → leave it in the greater region.

3 ≤ 4 → move 3 into the left region.

6 > 4 → leave it.

2 ≤ 4 → move 2 into the left region.

The ≤4 region is growing from the left.

# Partition around pivot 4 — finish

Continue scan: 7 > 4.

1 ≤ 4 → move 1 into the left region.

5 > 4.

Swap pivot 4 immediately after the ≤4 region.

One valid result: [3, 2, 1, 4, 7, 6, 5, 8]

Pivot 4 is now permanently in its final sorted position.

# Quick Sort — recurse on the partitions

### LEFT PARTITION

[3, 2, 1]

Choose a pivot and partition again.

Eventually → [1,2,3]

### RIGHT PARTITION

[7, 6, 5, 8]

Partition recursively.

Eventually → [5,6,7,8]

**Final result: [1,2,3,4,5,6,7,8]**

# Quick Sort best case

### BALANCED PIVOT

Pivot repeatedly splits the array into two nearly equal parts.

There are Θ(log n) recursive levels.

### PARTITION WORK

Partitioning costs Θ(n) per level across the subproblems.

$$T(n)=2T(n/2)+\Theta(n) \rightarrow \Theta(n \log n)$$

# Quick Sort worst case

### HIGHLY UNBALANCED PIVOT

Pivot repeatedly becomes the smallest or largest element.

Subproblem sizes become 0 and $n-1$.

### RECURRENCE

$T(n)=T(n-1)+\Theta(n)$

Expands to $\Theta(n^2)$.

**Worst-case time = $\Theta(n^2)$**

# Quick Sort average case

Typical pivots produce many reasonably balanced partitions even if every split is not exactly half.

Expected/average running time is Θ(n log n) under standard assumptions.

The exact analysis is more involved than the best-case recurrence.

Randomization is a common way to reduce dependence on input order.

**Average / expected behavior: Θ(n log n)**

COMPARE

# Same array — two Divide-and-Conquer strategies

| Feature | Merge Sort | Quick Sort |
|---|---|---|
| How it splits | By position: fixed halves | By value: pivot partition |
| Where main work happens | Combine / merge | Partition before recursion |
| Typical time | $\Theta(n \log n)$ | $\Theta(n \log n)$ average |
| Worst case | $\Theta(n \log n)$ | $\Theta(n^2)$ standard deterministic |
| Extra array memory | Usually $\Theta(n)$ | Typically small apart from recursion |

# Merge Sort vs Quick Sort

| Algorithm | Best | Average | Worst | Stable? |
|---|---|---|---|---|
| Merge Sort | Θ(n log n) | Θ(n log n) | Θ(n log n) | Usually yes |
| Quick Sort | Θ(n log n) | Θ(n log n) | $\Theta(n^2)$ | Usually no |

**Choice depends on guarantees, memory, stability and practical performance.**

# When would you prefer Merge Sort?

### GUARANTEE

You need guaranteed Θ(n log n) time.

### STABILITY

Stable sorting matters.

### DATA / STORAGE

Sequential access is natural, such as linked lists or external data.

# When would you prefer Quick Sort?

### ARRAYS

You want excellent practical average performance on arrays.

### MEMORY

In-place partitioning is valuable.

### PIVOT CONTROL

You can choose pivots carefully or randomly.

**Trade-off: poor pivot choices can still produce $\Theta(n^2)$ behavior.**

# Common mistakes

### MERGE SORT

Assuming both sides are already sorted before recursive calls.

Forgetting that merging contributes Θ(n) work at each level.

### QUICK SORT

Forgetting that the pivot is excluded from later recursive calls.

Claiming Quick Sort is always Θ(n log n).

### ANALYSIS

Writing a recurrence without explaining which operations produced its terms.

# Practice — trace both algorithms




1. Show the first two split levels of Merge Sort.
2. Merge [2,7,9] with [1,4,6] step by step.
3. For Quick Sort, choose 6 as the first pivot and show one complete partition.
4. State the resulting left and right Quick Sort subproblems.

# Practice — complexity questions

1. Why does Merge Sort remain Θ(n log n) on an already sorted input?
2. Write the best-case Quick Sort recurrence.
3. Write the worst-case Quick Sort recurrence.
4. Which algorithm gives the stronger worst-case guarantee?
5. Which algorithm normally uses less auxiliary array memory?

# Practice answers

1. Merge Sort still performs the same recursive splitting and linear merges.
2. $T(n)=2T(n/2)+\Theta(n) \rightarrow \Theta(n \log n)$.
3. $T(n)=T(n-1)+\Theta(n) \rightarrow \Theta(n^2)$.
4. Merge Sort gives $\Theta(n \log n)$ worst-case time.
5. Standard Quick Sort uses in-place partitioning; Merge Sort typically needs $\Theta(n)$ auxiliary array space.

# Lecture recap

| Idea | What to remember |
| --- | --- |
| Divide and Conquer | Divide → conquer → combine |
| Merge Sort | Fixed halves; $\Theta(n \log n)$ in all main cases |
| Quick Sort | Pivot partition; $\Theta(n \log n)$ average, $\Theta(n^2)$ worst |
| Recurrence | The terms must come from actual algorithmic work |

# We have reduced and divided problems

### DECREASE AND CONQUER

Reduce to ONE smaller instance.

n → n−1, n/2, or a variable-sized remainder.

### DIVIDE AND CONQUER

Split into MULTIPLE smaller instances.

Solve them and combine their results.

**But what if we do not want to reduce or divide the problem?**

# What if we transform the problem instead?

## ORIGINAL FORM

Unsorted data

Repeated searches would use linear search: Θ(n) per query.

## TRANSFORMED FORM

Sort the data first.

Then use binary search: Θ(log n) per query.

The transformation itself has a cost.

**Change the representation or formulation → solve the easier version.**

# Next: Transform and Conquer

### CONVENIENT INSTANCE

Transform the input into a form that is easier to process.

### DIFFERENT REPRESENTATION

Represent the same information in a structure better suited to the required operations.

### PROBLEM REDUCTION

Convert one problem into another problem we already know how to solve.

**Same goal: exploit structure instead of blindly doing more work.**